สัปดาห์ที่ 10: OpenAPI/Swagger & Developer Experience (DX)

สารบัญ

- วัตถุประสงค์การเรียนรู้ประจำสัปดาห์
- 1. Developer Experience (DX) คืออะไร และเหตุใดจึงสำคัญ
  - 1.1 นิยามและขอบเขต
  - 1.2 ต้นทุนของ DX ที่ไม่ดี
- 2. OpenAPI Specification 3.0
  - 2.1 OpenAPI คืออะไร
  - 2.2 องค์ประกอบสำคัญของไฟล์ OpenAPI
  - 2.3 ตัวอย่างไฟล์ OpenAPI: สร้างทีละ endpoint
  - 2.4 การระบุ Response ให้ครบทุกกรณี
- 3. Swagger UI: เชื่อมเอกสารเข้ากับโค้ดจริง
  - 3.1 ทำไมถึงสำคัญ: ปัญหาของเอกสารที่แยกจากโค้ด
  - 3.2 การติดตั้ง `swagger-ui-express`
  - 3.3 แนวทางรักษาความสอดคล้องระหว่างเอกสารกับโค้ด
- 4. Postman Collection และการจัดการ
  - 4.1 จากการเรียก Postman ทีละ request สู่ Collection ที่แชร์ได้
  - 4.2 การสร้างและจัดระเบียบ Collection
  - 4.3 การใช้ Environment Variable แทนค่าคงที่
  - 4.4 การเขียน Test Script ตรวจสอบ Response
  - 4.5 การ Export Collection และ Environment
  - 4.6 Newman: รัน Postman Collection แบบอัตโนมัติผ่าน Command Line
- 5. ความสัมพันธ์ระหว่างเอกสาร API กับคุณภาพซอฟต์แวร์โดยรวม
  - 5.1 เอกสารเป็นสัญญา (Contract) ระหว่างทีม
- สรุปเนื้อหาสำคัญ
- เอกสารอ้างอิง (Reference)
- คำถามท้ายบท

วัตถุประสงค์การเรียนรู้ประจำสัปดาห์

เมื่อเรียนจบเนื้อหาในสัปดาห์นี้ นิสิตจะสามารถ

1. อธิบายโครงสร้างของ OpenAPI Specification 3.0 และความสำคัญของการมีเอกสาร API ที่เป็นมาตรฐาน
2. เขียนไฟล์ OpenAPI Specification (`openapi.yaml`) บรรยาย endpoint, request/response schema และ status code ได้ถูกต้อง
3. ติดตั้งและตั้งค่า Swagger UI เพื่อให้เอกสาร API เข้าถึงได้ผ่านเบราว์เซอร์และสอดคล้องกับโค้ดจริง
4. สร้างและจัดการ Postman Collection สำหรับทดสอบ API อย่างเป็นระบบ
5. อธิบายแนวคิด Developer Experience (DX) และความสัมพันธ์ระหว่างเอกสาร API ที่ดีกับการนำ API ไปใช้งานจริงโดยทีมอื่นหรือระบบภายนอก

หัวข้อนี้สอดคล้องกับ CLO1, CLO6 และ CLO5

ดูคู่มืออ้างอิงคำสั่ง Swagger UI/Newman ที่ใช้บ่อย (เปิดดูเวลาเขียนโค้ด ไม่ต้องไล่อ่านเนื้อหาเต็ม) ได้ที่ ref-swagger-newman.md

---

1. Developer Experience (DX) คืออะไร และเหตุใดจึงสำคัญ

1.1 นิยามและขอบเขต

Developer Experience (DX) หมายถึงคุณภาพของประสบการณ์ที่นักพัฒนาคนอื่น (ทั้งภายในทีมและภายนอกทีม) ได้รับเมื่อต้องทำความเข้าใจและเรียกใช้งาน API ที่ผู้พัฒนาสร้างขึ้น เทียบเคียงได้กับแนวคิด User Experience (UX) แต่กลุ่มเป้าหมายคือนักพัฒนาแทนที่จะเป็นผู้ใช้ปลายทาง

API ที่มี DX ดีมีลักษณะร่วมกันดังนี้

คุณลักษณะ	ตัวอย่างรูปธรรม
เอกสารชัดเจนและตรงกับโค้ดจริง	Swagger UI แสดง endpoint ตรงกับที่ระบบรองรับจริง ไม่มี endpoint ที่ล้าสมัยหลงเหลืออยู่
ตัวอย่างใช้งานพร้อมทดสอบได้ทันที	Postman Collection ที่ import แล้วเรียกใช้ได้ทันทีโดยไม่ต้องคาดเดาค่า parameter
ข้อความ error สื่อความหมาย	คืน `400` พร้อม `{"error": {"code": "VALIDATION_ERROR", "message": "..."}}` แทนการคืน `500` ที่ไม่มีคำอธิบาย
Versioning ที่คาดเดาได้	ใช้ `/api/v1/...` ทำให้ทราบได้ทันทีว่า breaking change จะเกิดที่ `/api/v2/`

1.2 ต้นทุนของ DX ที่ไม่ดี

หากทีม Frontend หรือทีมภายนอกต้องเดาโครงสร้าง request/response จากการอ่านโค้ด backend โดยตรง หรือถามผู้พัฒนา API ทีละคำถามผ่านแชท จะเกิดต้นทุนต่อไปนี้

- เวลาพัฒนาฝั่ง Client ล่าช้าเนื่องจากต้องรอคำตอบ
- ความเสี่ยงที่ Client เข้าใจ schema ผิดและเกิด bug จากความไม่ตรงกัน (contract mismatch)
- ผู้พัฒนา API เดิมกลายเป็นคอขวด (bottleneck) เพราะต้องตอบคำถามซ้ำ ๆ

การลงทุนเขียนเอกสาร API ที่ดีจึงเป็นการลดต้นทุนการสื่อสารระยะยาว ไม่ใช่งานเสริมที่ทำหรือไม่ทำก็ได้

---

2. OpenAPI Specification 3.0

2.1 OpenAPI คืออะไร

OpenAPI Specification (เดิมชื่อ Swagger Specification) เป็นมาตรฐานเปิดสำหรับอธิบายโครงสร้างของ RESTful API ในรูปแบบที่ทั้งมนุษย์และเครื่องอ่านได้ (machine-readable) เขียนด้วย YAML หรือ JSON โดยครอบคลุม endpoint, HTTP method, parameter, request/response schema, authentication และอื่น ๆ

เนื่องจากเป็นรูปแบบมาตรฐาน เครื่องมือจำนวนมากจึงสามารถอ่านไฟล์ OpenAPI แล้วสร้างสิ่งต่อไปนี้โดยอัตโนมัติ

- หน้าเอกสารแบบโต้ตอบได้ (Swagger UI, Redoc)
- Client SDK สำหรับภาษาต่าง ๆ (code generation)
- ชุดทดสอบพื้นฐานจาก schema (contract testing)

2.2 องค์ประกอบสำคัญของไฟล์ OpenAPI

ไฟล์ OpenAPI หนึ่งไฟล์ประกอบด้วยส่วนหลักดังนี้

ส่วน	หน้าที่
`info`	ชื่อ คำอธิบาย และเวอร์ชันของ API
`servers`	URL ของ server ที่ใช้เรียก API จริง (แยก development/production ได้)
`paths`	รายการ endpoint ทั้งหมด พร้อม method, parameter, request body และ response ที่เป็นไปได้
`components.schemas`	นิยาม data model ที่ใช้ซ้ำในหลาย endpoint ผ่าน `$ref` เพื่อลดการเขียนซ้ำ

ส่วน	หน้าที่
`components.securitySchemes`	นิยามวิธีการยืนยันตัวตน เช่น JWT Bearer Token

หลักการสำคัญ: การใช้ `$ref` อ้างอิง schema ที่นิยามไว้ใน `components.schemas` แทนการเขียน schema ซ้ำในทุก endpoint ช่วยให้แก้ไข schema ครั้งเดียวมีผลกับทุกจุดที่อ้างอิง ลดความเสี่ยงที่เอกสารแต่ละ endpoint จะไม่ตรงกัน

2.3 ตัวอย่างไฟล์ OpenAPI: สร้างทีละ endpoint

ตัวอย่างชุดนี้นำ 5 ส่วนจากหัวข้อ 2.2 มาประกอบเป็นไฟล์จริงทีละ endpoint ใช้ field ตามโปรเจกต์ `student-api` จริง (`name` , `major` , `email`) ที่พัฒนามาตั้งแต่ต้นเทอม จึงนำไปใช้กับ `student-api` ได้ทันที แต่เมื่อเขียน `openapi.yaml` ของโปรเจกต์โครงงานของตนเอง (ระบบหนังสือ ระบบภาพยนตร์ ฯลฯ ตาม wk05-project-phase1.md) ห้ามคัดลอกชื่อ field เหล่านี้ไปใช้ ต้องเปลี่ยนเป็นชื่อ field ตามที่ endpoint จริงของโครงงานตนเองใช้งานอยู่

2.3.1 เริ่มจาก `info` , `servers` และ endpoint แรก (`GET /students`)

โครงไฟล์ขั้นต่ำสุดต้องมี `info` และ `servers` ก่อน จากนั้นเพิ่ม endpoint แรก คือการดึงรายการนิสิตพร้อม pagination

```yaml
openapi: 3.0.3
info:
  title: Student API
  description: API สำหรับจัดการข้อมูลนิสิต
  version: 1.0.0
servers:
  - url: http://localhost:3000/api/v1
    description: Development server
paths:
  /students:
    get:
      summary: ดึงรายการนิสิตทั้งหมด
      tags:
        - Students
      parameters:
        - name: page
          in: query
          schema:
            type: integer
            default: 1
          description: หมายเลขหน้าที่ต้องการ
```

```yaml
        - name: limit
          in: query
          schema:
            type: integer
            default: 10
          description: จำนวนรายการต่อหน้า
      responses:
        "200":
          description: สำเร็จ คืนรายการนิสิตพร้อมข้อมูล pagination
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/StudentListResponse"
        "401":
          description: ไม่ได้แนบ token หรือ token ไม่ถูกต้อง
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/ErrorResponse"
components:
  schemas:
    Student:
      type: object
      properties:
        id:
          type: integer
          example: 1
        name:
          type: string
        major:
          type: string
        email:
          type: string
    StudentListResponse:
      type: object
      properties:
        data:
          type: array
          items:
            $ref: "#/components/schemas/Student"
        pagination:
          type: object
          properties:
            page:
              type: integer
            limit:
```

```
            type: integer
          totalItems:
            type: integer
          totalPages:
            type: integer
    ErrorResponse:
      type: object
      properties:
        error:
          type: object
          properties:
            code:
              type: string
              example: "VALIDATION_ERROR"
            message:
              type: string
              example: "name is required"
```

สังเกตว่า response `200` และ `401` อ้างอิง schema ผ่าน `$ref` ไปยัง `components.schemas` ที่นิยามไว้ด้านล่าง ไม่ได้เขียน schema ซ้ำในบรรทัดเดียวกัน

2.3.2 เพิ่ม `POST /students` (สร้างนิสิตใหม่)

เพิ่ม method `post` ใต้ path `/students` เดิม พร้อม schema สำหรับ request body (`StudentInput`) และ response (`StudentResponse`) เนื่องจาก endpoint นี้ต้องยืนยันตัวตน จึงต้องมี `security: [{ bearerAuth: [] }]` และนิยาม `securitySchemes` เพิ่มใน `components`

yaml

```yaml
paths:
  /students:
    get:
      # ... เหมือนหัวข้อ 2.3.1 ...
    post:
      summary: สร้างนิสิตใหม่
      tags:
        - Students
      security:
        - bearerAuth: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/StudentInput"
      responses:
        "201":
```

```yaml
          description: สร้างสำเร็จ
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/StudentResponse"
        "400":
          description: ข้อมูลไม่ผ่านการตรวจสอบ (validation)
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/ErrorResponse"
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  schemas:
    StudentInput:
      type: object
      required:
        - name
        - major
        - email
      properties:
        name:
          type: string
          example: "สมชาย ใจดี"
        major:
          type: string
          example: "วิทยาการคอมพิวเตอร์"
        email:
          type: string
          example: "somchai@example.com"
    StudentResponse:
      type: object
      properties:
        data:
          $ref: "#/components/schemas/Student"
    # Student, StudentListResponse, ErrorResponse เหมือนหัวข้อ 2.3.1
```

2.3.3 เพิ่ม `/students/{id}` (ดึงข้อมูลตาม id)

เพิ่ม path ใหม่ `/students/{id}` ที่มี path parameter `id` และ response `404` สำหรับกรณีไม่พบข้อมูล

```yaml
paths:
  /students:
    get:
      # ... เหมือนหัวข้อ 2.3.1 ...
    post:
      # ... เหมือนหัวข้อ 2.3.2 ...
  /students/{id}:
    get:
      summary: ดึงข้อมูลนิสิตตาม id
      tags:
        - Students
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        "200":
          description: สำเร็จ
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/StudentResponse"
        "404":
          description: ไม่พบนิสิตตาม id ที่ระบุ
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/ErrorResponse"
```

2.3.4 ไฟล์ฉบับเต็มเมื่อรวมทั้ง 3 endpoint

ไฟล์ `openapi.yaml` ที่รวมทุกส่วนจากหัวข้อ 2.3.1-2.3.3 เข้าด้วยกัน (ไม่มีคอมเมนต์ย่อ) มีหน้าตาดังนี้

```yaml
openapi: 3.0.3
info:
  title: Student API
  description: API สำหรับจัดการข้อมูลนิสิต
  version: 1.0.0
servers:
  - url: http://localhost:3000/api/v1
    description: Development server
paths:
```

```yaml
  /students:
    get:
      summary: ดึงรายการนิสิตทั้งหมด
      tags:
        - Students
      parameters:
        - name: page
          in: query
          schema:
            type: integer
            default: 1
          description: หมายเลขหน้าที่ต้องการ
        - name: limit
          in: query
          schema:
            type: integer
            default: 10
          description: จำนวนรายการต่อหน้า
      responses:
        "200":
          description: สำเร็จ คืนรายการนิสิตพร้อมข้อมูล pagination
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/StudentListResponse"
        "401":
          description: ไม่ได้แนบ token หรือ token ไม่ถูกต้อง
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/ErrorResponse"
    post:
      summary: สร้างนิสิตใหม่
      tags:
        - Students
      security:
        - bearerAuth: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/StudentInput"
      responses:
        "201":
          description: สร้างสำเร็จ
```

```yaml
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/StudentResponse"
        "400":
          description: ข้อมูลไม่ผ่านการตรวจสอบ (validation)
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/ErrorResponse"
  /students/{id}:
    get:
      summary: ดึงข้อมูลนิสิตตาม id
      tags:
        - Students
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        "200":
          description: สำเร็จ
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/StudentResponse"
        "404":
          description: ไม่พบนิสิตตาม id ที่ระบุ
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/ErrorResponse"
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  schemas:
    StudentInput:
      type: object
      required:
        - name
        - major
```

```yaml
      - email
    properties:
      name:
        type: string
        example: "สมชาย ใจดี"
      major:
        type: string
        example: "วิทยาการคอมพิวเตอร์"
      email:
        type: string
        example: "somchai@example.com"
  Student:
    type: object
    properties:
      id:
        type: integer
        example: 1
      name:
        type: string
      major:
        type: string
      email:
        type: string
  StudentResponse:
    type: object
    properties:
      data:
        $ref: "#/components/schemas/Student"
  StudentListResponse:
    type: object
    properties:
      data:
        type: array
        items:
          $ref: "#/components/schemas/Student"
      pagination:
        type: object
        properties:
          page:
            type: integer
          limit:
            type: integer
          totalItems:
            type: integer
          totalPages:
            type: integer
```

```yaml
    ErrorResponse:
      type: object
      properties:
        error:
          type: object
          properties:
            code:
              type: string
              example: "VALIDATION_ERROR"
            message:
              type: string
              example: "name is required"
```

2.4 การระบุ Response ให้ครบทุกกรณี

จุดที่มักถูกมองข้ามคือการระบุเฉพาะกรณีสำเร็จ (`200` / `201`) โดยไม่ระบุกรณีผิดพลาด เอกสาร OpenAPI ที่ดีต้องระบุ status code ที่เป็นไปได้ทั้งหมด เช่น `400` (validation), `401` (ไม่ได้ยืนยันตัวตน), `403` (ไม่มีสิทธิ์ตาม RBAC), `404` (ไม่พบทรัพยากร) และ `409` (ข้อมูลซ้ำ) ตามความเหมาะสมของแต่ละ endpoint

ตัวอย่างเช่น `DELETE /students/{id}` ที่จำกัดสิทธิ์เฉพาะ role `admin` ตาม RBAC ต้องระบุ response `403` ไว้ด้วย ไม่ใช่แค่ `200` และ `404`

```yaml
/students/{id}:
  delete:
    summary: ลบข้อมูลนิสิตตาม id
    tags:
      - Students
    security:
      - bearerAuth: []
    parameters:
      - name: id
        in: path
        required: true
        schema:
          type: integer
    responses:
      "200":
        description: ลบสำเร็จ
      "403":
        description: ไม่มีสิทธิ์ลบข้อมูล (ต้องเป็น role admin)
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/ErrorResponse"
      "404":
```

```
        description: ไม่พบนิสิตตาม id ที่ระบุ
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/ErrorResponse"
```

---

3. Swagger UI: เชื่อมเอกสารเข้ากับโค้ดจริง

Swagger UI คือเครื่องมือโอเพนซอร์สที่อ่านไฟล์ OpenAPI Specification (`openapi.yaml` ที่เขียนไว้ในหัวข้อ 2) แล้วแปลงเป็นหน้าเว็บเอกสาร API แบบโต้ตอบได้ (interactive) โดยอัตโนมัติ โดยไม่ต้องเขียนหน้าเว็บเอง หน้าที่ได้จะแสดงรายการ endpoint ทั้งหมดจัดกลุ่มตาม `tags` พร้อมปุ่ม **Try it out** ที่ให้กรอกค่าพารามิเตอร์แล้วยิง HTTP request ไปยัง server จริงเพื่อดู response ได้ทันทีจากหน้าเว็บ เทียบเคียงได้กับการใช้ Postman แต่แสดงผลอยู่ในหน้าเดียวกับตัวเอกสารเอง

3.1 ทำไมถึงสำคัญ: ปัญหาของเอกสารที่แยกจากโค้ด

เอกสาร API ที่เขียนแยกต่างหาก (เช่น ไฟล์ Word หรือ Google Docs) มีความเสี่ยงสูงที่จะล้าสมัย (stale documentation) เนื่องจากไม่มีกลไกบังคับให้ปรับปรุงพร้อมกับโค้ด Swagger UI แก้ปัญหานี้บางส่วนโดยให้เอกสารอยู่ในรูปแบบไฟล์ที่ผูกกับ repository เดียวกับโค้ด และแสดงผลผ่าน endpoint ของแอปพลิเคชันเอง

3.2 การติดตั้ง `swagger-ui-express`

```
npm install swagger-ui-express js-yaml
```

```javascript
// app.js
const express = require("express");
const swaggerUi = require("swagger-ui-express");
const YAML = require("js-yaml");
const fs = require("fs");
const path = require("path");

const app = express();

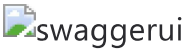
const openapiDocument = YAML.load(
  fs.readFileSync(path.join(__dirname, "openapi.yaml"), "utf8"),
);

app.use("/api-docs", swaggerUi.serve, swaggerUi.setup(openapiDocument));

// ... route อื่น ๆ ทั้งหมดตามที่มีอยู่เดิม ...
```

```
module.exports = app;
```

เมื่อรันเซิร์ฟเวอร์และเปิด `http://localhost:3000/api-docs` จะเห็นหน้าเอกสารแบบโต้ตอบได้ (interactive) ที่สามารถกดทดลองเรียก endpoint ได้ทันทีจากหน้าเว็บ โดยไม่ต้องเปิด Postman


swaggerui

ข้อควรระวังด้าน security: โค้ดตัวอย่างข้างต้น mount `/api-docs` แบบเปิดสาธารณะโดยไม่มีการป้องกันใด ๆ ซึ่งเหมาะสำหรับ development เท่านั้น ในระบบ production ไม่ควร expose หน้าเอกสาร API แบบนี้ให้ทุกคนเข้าถึงได้อย่างอิสระ เพราะเป็นการเปิดเผยโครงสร้าง endpoint ทั้งหมดของระบบให้ผู้ไม่หวังดี แนวทางที่ควรทำอย่างน้อยหนึ่งอย่างคือ (1) เปิด `/api-docs` เฉพาะเมื่อไม่ใช่ production

```javascript
if (process.env.NODE_ENV !== "production") {
  app.use("/api-docs", swaggerUi.serve, swaggerUi.setup(openapiDocument));
}
```

หรือ (2) ใส่ authentication middleware ปกป้อง route นี้เช่นเดียวกับ route อื่นที่ต้องยืนยันตัวตน

3.3 แนวทางรักษาความสอดคล้องระหว่างเอกสารกับโค้ด

แนวทาง	รายละเอียด
ปรับปรุง `openapi.yaml` ทุกครั้งที่เพิ่ม/แก้ไข endpoint	กำหนดเป็นส่วนหนึ่งของ definition of done ของแต่ละ feature
ตรวจสอบด้วยตนเองก่อน commit	เปิด `/api-docs` เปรียบเทียบกับพฤติกรรมจริงของ endpoint ที่เพิ่งแก้ไข
Code review ตรวจสอบเอกสารร่วมกับโค้ด	ผู้ตรวจ pull request ตรวจสอบว่า `openapi.yaml` ถูกอัปเดตพร้อมกับ route ที่เปลี่ยนแปลงหรือไม่

รายวิชานี้เน้นแนวทางแรกสองข้อสำหรับระดับโครงงานรายบุคคล ส่วนการตรวจสอบความสอดคล้องอัตโนมัติด้วยเครื่องมือขั้นสูง (เช่น contract testing ด้วย `express-openapi-validator` — สืบค้นจาก https://github.com/cdimascio/express-openapi-validator) เป็นหัวข้อขยายที่นอกเหนือขอบเขตของแล็บสัปดาห์นี้

---

4. Postman Collection และการจัดการ

4.1 จากการเรียก Postman ทีละ request สู่ Collection ที่แชร์ได้

การทดสอบ API ด้วย Postman ที่ผ่านมาเป็นการเรียกทีละ request แยกกัน Postman Collection คือการรวบรวม request ทั้งหมดของโปรเจกต์ไว้เป็นไฟล์เดียวที่จัดหมวดหมู่ได้ พร้อมตัวแปร (environment variable) และตัวอย่าง response ที่บันทึกไว้ ทำให้สมาชิกในทีมหรือผู้ประเมินภายนอกสามารถ import แล้วทดสอบ API ได้ทันทีโดยไม่ต้องตั้งค่าใหม่ทั้งหมด

4.2 การสร้างและจัดระเบียบ Collection

โครงสร้าง Collection ที่แนะนำสำหรับโปรเจกต์ `student-api`

```
student-api (Collection)
├── Auth
│   ├── POST Register
│   ├── POST Login
│   └── GET Me
└── Students
    ├── GET List (with pagination)
    ├── POST Create
    ├── PUT Update
    └── DELETE Remove
```

4.3 การใช้ Environment Variable แทนค่าคงที่

แทนที่จะเขียน URL และ token ตายตัวในแต่ละ request ให้ใช้ตัวแปรของ Postman

ตัวแปร	ตัวอย่างค่า	จุดประสงค์
`{{baseUrl}}`	`http://localhost:3000/api/v1`	เปลี่ยนสภาพแวดล้อม (dev/staging) โดยไม่ต้องแก้ทุก request
`{{authToken}}`	ค่าที่ได้จาก response ของ `POST /auth/login`	ใช้ซ้ำในทุก request ที่ต้องยืนยันตัวตน

Postman รองรับการเขียนสคริปต์ในแท็บ **Tests** ของ request `POST /auth/login` เพื่อบันทึก token เข้าตัวแปรอัตโนมัติ

```javascript
// Tests tab ของ request POST /auth/login
const response = pm.response.json();
pm.environment.set("authToken", response.token);
```

จากนั้น request อื่นที่ต้องยืนยันตัวตนสามารถตั้งค่า Header เป็น `Authorization: Bearer {{authToken}}` ได้โดยไม่ต้องคัดลอก token ด้วยตนเองทุกครั้ง

4.4 การเขียน Test Script ตรวจสอบ Response

Collection ที่มีแต่ request โดยไม่มีการตรวจสอบผลลัพธ์ยังไม่ต่างจากการเรียก Postman แบบเดิมมากนัก เพราะต้องอ่าน response ด้วยตาทุกครั้งว่าค่าที่ได้ถูกต้องหรือไม่ Postman จึงรองรับการเขียนสคริปต์ในแท็บ **Tests** ของแต่ละ request เพื่อตรวจสอบอัตโนมัติ เทียบเคียงได้กับ `assert` ใน Jest/Supertest ที่เรียนมาในสัปดาห์ที่ 9 เพียงแต่ทำงานที่ระดับ HTTP response แทนที่จะเป็นโค้ดในโปรเจกต์

```javascript
// GET Students - Tests tab
pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
});

pm.test("Response has pagination info", function () {
  const body = pm.response.json();
  pm.expect(body).to.have.property("pagination");
});
```

ทุก request ใน Collection ควรมี Test script อย่างน้อยตรวจสอบ status code ที่ถูกต้อง และ request ที่ทดสอบกรณีผิดพลาด (เช่น เรียก `GET /students/{id}` ด้วย id ที่ไม่มีอยู่จริง) ก็ควรมี request แยกต่างหาก พร้อม Test script ตรวจสอบว่าได้ `404` กลับมา ไม่ใช่ทดสอบเฉพาะกรณีสำเร็จเท่านั้น

4.5 การ Export Collection และ Environment

```
Collection > ... (เมนู) > Export > Collection v2.1 (recommended) → บันทึกเป็น
postman_collection.json
Environment > ... (เมนู) > Export → บันทึกเป็น environment.json
```

ต้อง export ทั้งสองไฟล์คู่กัน เพราะ `postman_collection.json` เก็บเฉพาะรายการ request และ Test script ส่วนค่าตัวแปรอย่าง `{{baseUrl}}` อยู่ใน `environment.json` แยกต่างหาก หากส่งแค่ไฟล์ Collection แล้วไม่ส่ง Environment ผู้ที่ import จะเรียก request ไม่ได้เพราะไม่มีค่าตัวแปรให้แทนที่ ทั้งสองไฟล์เป็น JSON ที่เก็บไว้ใน repository ของโปรเจกต์ได้ เช่นเดียวกับ `openapi.yaml` ในหัวข้อ 2 ทำให้ทีมหรือผู้ประเมินภายนอก clone โปรเจกต์แล้ว import กลับเข้า Postman ได้ทันทีโดยไม่ต้องตั้งค่าใหม่ทั้งหมด

4.6 Newman: รัน Postman Collection แบบอัตโนมัติผ่าน Command Line

Newman คือ Command-line Collection Runner ของ Postman ทำให้รัน Collection ที่สร้างไว้ได้โดยไม่ต้องเปิดโปรแกรม Postman ซึ่งเปิดทางให้นำไปรันอัตโนมัติร่วมกับ CI/CD ได้ในอนาคต

```
npm install -g newman
newman run postman_collection.json -e environment.json
```

ผลลัพธ์ที่คาดหวัง

```
Student API

→ POST Register
  POST http://localhost:3000/api/v1/auth/register [201 Created, 512B, 45ms]
  ✓  Status code is 201

→ POST Login
  POST http://localhost:3000/api/v1/auth/login [200 OK, 380B, 32ms]
  ✓  Status code is 200
  ✓  Response has token

┌─────────────────────────┬────────────────────┬────────────────────┐
│                         │          executed  │             failed │
├─────────────────────────┼────────────────────┼────────────────────┤
│              iterations │                  1 │                  0 │
│                requests │                  7 │                  0 │
│            test-scripts │                  7 │                  0 │
│              assertions │                 14 │                  0 │
└─────────────────────────┴────────────────────┴────────────────────┘
```

Newman จึงทำหน้าที่คล้ายกับ Supertest แต่ทำงานที่ระดับ HTTP จริงผ่าน Collection ที่ไม่ผูกกับภาษาโปรแกรมใดภาษาหนึ่ง เหมาะสำหรับทดสอบ API แบบ black-box จากมุมมองของผู้ใช้ภายนอกระบบ

---

5. ความสัมพันธ์ระหว่างเอกสาร API กับคุณภาพซอฟต์แวร์โดยรวม

5.1 เอกสารเป็นสัญญา (Contract) ระหว่างทีม

เมื่อทีม Backend และ Frontend พัฒนาแยกกัน OpenAPI Specification ทำหน้าที่เป็น “สัญญา” (contract) ที่ทั้งสองฝ่ายตกลงร่วมกันก่อนเริ่มพัฒนา แนวทางนี้เรียกว่า **API-First Design** ซึ่งต่างจากการเขียนโค้ดก่อนแล้วค่อยเขียนเอกสารตามทีหลัง (code-first) โดยข้อดีของ API-First คือทีม Frontend เริ่มพัฒนาคู่ขนานได้ทันทีโดยใช้ mock server ที่สร้างจาก OpenAPI Specification เช่นเครื่องมือ Prism (https://github.com/stoplightio/prism) ซึ่งอ่านไฟล์ `openapi.yaml` แล้วจำลอง response ตาม schema ที่นิยามไว้โดยอัตโนมัติ

```
npx @stoplight/prism-cli mock openapi.yaml
```

คำสั่งนี้เป็นเพียงตัวอย่างให้รู้จักแนวคิด ไม่ใช่ขั้นตอนที่ต้องทำในแล็บสัปดาห์นี้ ทำให้ทีม Frontend ไม่ต้องรอ Backend เสร็จสมบูรณ์ก่อน

เนื้อหาสัปดาห์นี้จึงเป็นการรวบรวม (consolidate) การออกแบบและพฤติกรรมจริงของ API ที่สร้างมาตลอดภาคการศึกษาให้อยู่ในรูปแบบเอกสารเดียวที่ตรวจสอบได้

สรุปเนื้อหาสำคัญ

1. Developer Experience (DX) คือคุณภาพประสบการณ์ของนักพัฒนาที่ต้องเรียกใช้ API ที่ผู้พัฒนาสร้างขึ้น เอกสารที่ดีช่วยลดต้นทุนการสื่อสารและความเสี่ยงจาก contract mismatch
2. OpenAPI Specification 3.0 เป็นมาตรฐานเปิดสำหรับอธิบาย endpoint, schema และ authentication ในรูปแบบที่เครื่องมืออ่านต่อได้ ประกอบด้วย `info`, `servers`, `paths` และ `components`
3. Swagger UI แสดงผล OpenAPI Specification เป็นหน้าเอกสารโต้ตอบได้ผ่าน endpoint ของแอปพลิเคชันเอง ช่วยลดความเสี่ยงที่เอกสารจะล้าสมัยเมื่อเทียบกับเอกสารที่แยกไฟล์ต่างหาก
4. Postman Collection รวบรวม request ทั้งหมดของโปรเจกต์พร้อม environment variable ให้แชร์และทดสอบซ้ำได้ง่าย ส่วน Newman เปิดทางให้รัน Collection ผ่าน command line แบบอัตโนมัติ
5. API-First Design ใช้ OpenAPI Specification เป็นสัญญาระหว่างทีม Backend และ Frontend ทำให้พัฒนาคู่ขนานกันได้
6. เนื้อหาสัปดาห์นี้เป็นการรวบรวมการออกแบบ API ที่เรียนมาก่อนหน้าให้อยู่ในรูปแบบเอกสารมาตรฐานเดียว และเป็นส่วนหนึ่งของโครงงานระยะ 2 (Docs/DX)

ฝึกปฏิบัติตามเนื้อหาข้างต้นได้ที่ wk10-lab.md ส่วนเกณฑ์การให้คะแนนโครงงานระยะ 2 (Docs/DX) ที่ต้องส่งจริงอยู่ที่ wk10-project-phase2.md

---

เอกสารอ้างอิง (Reference)

1. OpenAPI Initiative. *OpenAPI Specification v3.0.3*. สืบค้นจาก https://swagger.io/specification
2. Swagger. *Swagger UI*. สืบค้นจาก https://swagger.io/tools/swagger-ui
3. Postman. *Postman Learning Center*. สืบค้นจาก https://learning.postman.com
4. Postman. *Newman: Postman Command Line Collection Runner*. สืบค้นจาก https://github.com/postmanlabs/newman
5. Express.js Official Documentation. สืบค้นจาก https://expressjs.com

---

คำถามท้ายบท

1. จงอธิบายความหมายของ Developer Experience (DX) และยกตัวอย่างผลกระทบที่เกิดขึ้นเมื่อ API มี DX ไม่ดี
2. OpenAPI Specification ประกอบด้วยส่วนหลักใดบ้าง และแต่ละส่วนทำหน้าที่อะไร
3. เพราะเหตุใดการใช้ `$ref` อ้างอิง schema ใน `components.schemas` จึงดีกว่าการเขียน schema ซ้ำในทุก endpoint
4. จงเขียนโครงสร้าง OpenAPI YAML สำหรับ endpoint `DELETE /api/v1/students/{id}` ให้ครบทั้งกรณีสำเร็จ (`200`) และกรณีไม่พบข้อมูล (`404`)
5. Swagger UI ช่วยลดปัญหาเอกสารล้าสมัย (stale documentation) ได้อย่างไร เมื่อเทียบกับการเขียนเอกสารแยกไฟล์ต่างหาก
6. Postman Collection แตกต่างจากการทดสอบ Postman แบบเรียกทีละ request อย่างไร และเหตุใดการใช้ Environment Variable จึงสำคัญ
7. Newman ทำหน้าที่คล้ายกับเครื่องมือใดที่เรียนในสัปดาห์ที่ 9 และแตกต่างกันอย่างไรในแง่ระดับของการทดสอบ (code-level เทียบกับ HTTP-level)
8. จงอธิบายแนวคิด API-First Design และประโยชน์ต่อการทำงานร่วมกันระหว่างทีม Backend และ Frontend